CS370 Fall 2024 Final Report

Model Performance and Feature Importance

My individual model performance and feature importance analysis results can be found in a separate report on the repository. However, I will utilize this section to describe more about how I implemented my individual codebase. I developed my implementation of my various models for comparison by using the WindowGenerator class from the TensorFlow tutorial as my foundation. This was important for utilizing the way the WindowGenerator split the datasets into windows and batched them together. I also followed the tutorial's method of creating wind vectors from the two variables wind speed and wind direction. After deciding to use the WindowGenerator class, I developed helper functions that would take in the number of features and output window size as parameters to build my various models including CNN, RNN, Linear, and LSTM models. I developed these models using a mix of inspiration from the TensorFlow tutorial and the existing project codebase.

I created a dictionary of predetermined configurations that are further described in the more detailed report. Each configuration changed the input window size, output window size, and selected label feature. I then compared all of the models for each configuration and printed the results out while also including them in CSV files. The same thing was done for my feature importance code except I only used one configuration and compared the various models while dropping 1 feature at a time.

Script Integration

After completing the scripts for determining the models that performed best across the various configurations and finding the features with the most impact on the models, Alex and I worked on integrating our codebases to generate a singular script to take in command line arguments and run our model comparisons and various methods of feature importance on cloud machines for long periods of time.

We decided to use my original codebase as the foundation of our integration and began by first creating a separate file for our helper methods. The documentation is below. After creating the helper functions, Alex and I then cleaned up the main script and began testing all of our models on the various configurations we had originally created on a singular station of data. We referenced both the existing overall codebase of the project and the TensorFlow tutorial whenever we ran into difficulties. After testing with set configurations, the script was then altered to take in command-line arguments for more flexibility and versatility.

The script takes in command line arguments for the features to predict, input window size, output window size, and number of stations to test and train on. The script automatically loads the station CSVs into data frames before automatically creating CSVs for loss history, feature importance, and model comparison results for the given configuration determined by the command line arguments.

Similar to my original model performance and feature importance analysis codebase, the various models are stored in a dictionary and created by the helper functions below. All of the models are then trained and evaluated before storing the metrics MAE, MSE, and MAPE in the model comparison CSV.

The feature importance portion of the script expands upon my individual method of dropping singular features with two new methods. My original method involved rebuilding models by dropping a feature at a time and keeping the rest. Whichever feature dropped had the highest MAE or MSE would be deemed the most important model. The new methods went in a different direction that involved building up models from singular features. The best feature would be defined as the singular feature model with the best metric result. This feature would be kept and then the models would be rebuilt with the rest of the features added individually. The best feature combination would be kept and this would be iteratively repeated until the model stopped improving.

All of these models were saved in .keras format and relevant results were stored in CSVs.

Model Functions

Each function defines a specific model architecture for time series forecasting.

`baseline(label_width, num_features)`

- **Description**: Predicts the last value (`swc_5`) of the sequence using a lambda function.
- **Output**: A simple sequential model with one lambda layer.

`linear(label_width, num_features)`

- **Description**: A linear model that takes the last input value and outputs predictions for all features using a dense layer.
- **Output**: A sequential model with lambda, dense, and reshape layers.

`dense(label_width, num_features)`

- **Description**: A fully connected dense neural network that predicts the output sequence.
- **Layers**:
  - Lambda layer.
  - Dense layer with 512 units and ReLU activation.
  - Dense layer for prediction followed by reshaping.
- **Output**: A sequential model.

`simple_rnn(label_width, num_features)`

- **Description**: Implements a SimpleRNN architecture for time series prediction.
- **Layers**:
  - SimpleRNN with 64 units.
  - Dense and reshape layers for the output.
- **Output**: A sequential RNN-based model.

`cnn(label_width, num_features, CONV_WIDTH)`

- **Description**: A convolutional neural network that uses a 1D convolution layer for feature extraction.
- **Layers**:
  - Lambda for slicing.
  - Conv1D with 256 filters and ReLU activation.
  - Dense and reshape layers for the output.
- **Output**: A sequential CNN model.

`lstm(label_width, num_features)`

- **Description**: Implements an LSTM network for sequential data.
- **Layers**:
  - LSTM with 64 units.
  - Dense and reshape layers for the output.
- **Output**: A sequential LSTM-based model.

`autoregressive(label_width, num_features)`

- **Description**: An autoregressive model using dense layers and global average pooling.
- **Layers**:
  - Dense layer with 128 units.
  - Global average pooling.
  - Dense and reshape layers for the output.
- **Output**: A sequential autoregressive model.

`bi_lstm(label_width, num_features)`

- **Description**: A bidirectional LSTM model for sequence-to-sequence prediction.
- **Layers**:
  - Three Bidirectional LSTM layers.
  - Dense layers for feature extraction and prediction.
- **Output**: A sequential Bi-LSTM model.

`bi_lstm_attention(label_width, num_features)`

- **Description**: Combines bidirectional LSTM with self-attention for enhanced feature learning.
- **Layers**:
  - Bidirectional LSTM layers.
  - Self-attention layer (`SeqSelfAttention`).
  - Dense and reshape layers for the output.
- **Output**: A sequential Bi-LSTM model with attention.

`lstm_attention(label_width, num_features)`

- **Description**: Combines LSTM with self-attention for improved time series forecasting.
- **Layers**:
  - Two LSTM layers.
  - Self-attention layer (`SeqSelfAttention`).

    - Dense and reshape layers for the output.
- **Output**: A sequential LSTM model with attention.

Data Loading and Preprocessing Functions

`load_data(station, data_path)`

- **Description**: Loads a specific station's dataset, ensures no duplicate timestamps, and optionally limits the data size.
- **Parameters**:
    - `station`: The station number.
    - `data_path`: The file path to the datasets.
- **Output**: A cleaned DataFrame.

`load_all_data(data_path, stations_amt)`

- **Description**: Loads data for multiple stations and concatenates them into one DataFrame.
- **Parameters**:
    - `data_path`: File path to datasets.
    - `stations_amt`: Number of stations to load.
- **Output**: A dictionary containing the concatenated DataFrame.

`preprocess(df)`

- **Description**: Performs data preprocessing, including:
    - Dropping unnecessary columns (`Latitude`, `Longitude`).
    - Calculating wind components (`Wx`, `Wy`) from speed and direction.
    - Adding cyclical time features (`Day sin`, `Day cos`, `Year sin`, `Year cos`).
- **Input**: A DataFrame.
- **Output**: A preprocessed DataFrame.

`normalize(data)`

- **Description**: Standardizes the data by subtracting the mean and dividing by the standard deviation.
- **Input**: A DataFrame.
- **Output**: A normalized DataFrame.

Documentation for the `WindowGenerator` Class

The `WindowGenerator` class provides a structured approach for handling time-series data windows for machine learning models. It allows the creation of input-output windows from sequential data, making it suitable for tasks like forecasting or time-series prediction. Below is a detailed explanation of the class methods and attributes:

Attributes

- **`input_width`**: The number of time steps used as input.
- **`label_width`**: The number of time steps to predict (output).
- **`shift`**: The number of time steps between the last input and the first label.
- **`train_df`**: DataFrame containing training data.
- **`val_df`**: DataFrame containing validation data.
- **`test_df`**: DataFrame containing test data.
- **`label_columns`**: List of specific columns used as labels. If `None`, all columns are used.
- **`label_columns_indices`**: Dictionary mapping `label_columns` to their indices.
- **`column_indices`**: Dictionary mapping all column names to their indices.
- **`total_window_size`**: Total number of time steps in each window (input + shift).
- **`input_slice`**: Slice object for extracting input data from the window.
- **`input_indices`**: Numpy array of indices for the input section of the window.
- **`labels_slice`**: Slice object for extracting label data from the window.
- **`label_indices`**: Numpy array of indices for the label section of the window.

Methods

`__init__`

Initializes the window generator with specified parameters, such as input and label widths, shift, and data splits (train, validation, and test).

`split_window(features)`

- **Input**: A batch of data containing both inputs and labels.
- **Output**:
  - `inputs`: Extracted input features of shape `[batch_size, input_width, num_features]`.
  - `labels`: Extracted label features of shape `[batch_size, label_width, num_features]`.
- If `label_columns` is specified, only those columns are included in the `labels`.

`make_dataset(data)`

- **Input**: A DataFrame or array of time-series data.
- **Output**: A TensorFlow dataset object with time-series windows.
- Converts the data into a format suitable for feeding into machine learning models, including shuffling and batching.

Properties

- **`train`**: Returns the training dataset as a TensorFlow dataset.
- **`val`**: Returns the validation dataset as a TensorFlow dataset.
- **`test`**: Returns the test dataset as a TensorFlow dataset.

`plot(model_name, config, model=None, plot_col='SWC_5', max_subplots=3)`

- **Purpose**: Visualizes the inputs, labels, and (optionally) model predictions for the specified column (`plot_col`).
- **Parameters**:
  - `model_name`: Name of the model (used in the plot title).
  - `config`: Configuration or description of the model (used in the plot title).
  - `model`: Trained model for generating predictions.
  - `plot_col`: Column name to plot (default is `'SWC_5'`).
  - `max_subplots`: Maximum number of subplots to display.
- **Output**: A matplotlib plot comparing inputs, labels, and predictions.

Helper Function

`create_window`

- **Purpose**: Factory function for creating a `WindowGenerator` object.
- **Parameters**:
  - `input_width`: Width of the input window.
  - `label_width`: Width of the label window.
  - `shift`: Time steps between the end of the input and the start of the label.
  - `train_df`, `val_df`, `test_df`: Training, validation, and test datasets as DataFrames.
  - `label_columns`: List of columns used as labels.
- **Output**: An instance of the `WindowGenerator` class.

`train_and_evaluate_models`

Trains and evaluates multiple models using a specified configuration and datasets.

Parameters:

- `station` (str): Identifier for the station being evaluated.
- `config` (dict): Model configuration, including input/output lengths and target features.
- `models` (dict): Dictionary of models to train, where keys are model names and values are model objects.
- `train_df`, `val_df`, `test_df` (pd.DataFrame): Training, validation, and test datasets.
- `model_dir` (str): Directory to save trained models.
- `model_file` (str): CSV file for storing model results.
- `loss_file` (str): CSV file for storing loss history.

Returns:

- `performance` (dict): Test performance metrics for all models.
- `val_performance` (dict): Validation performance metrics for all models.
- `history_dicts` (dict): Training history for each model.

`plot_performance`

Plots validation and test performance (MAE) for all models.

Parameters:

- `multi_val_performance` (dict): Validation performance metrics for all models.
- `multi_test_performance` (dict): Test performance metrics for all models.
- `title` (str): Title for the plot (default: 'Model Comparison').

Returns:

- None (displays a bar chart).

`print_model_summaries`

Prints a summary of model performance across all configurations.

Parameters:

- `models` (dict): Dictionary of model names and objects.
- `all_losses` (dict): Performance metrics for all configurations and models.

Returns:

- None (outputs summary to the console).

`create_csv`

Initializes a CSV file with a header for storing model results.

Parameters:

- `csv_filename` (str): Name of the CSV file to create.

Returns:

- None (creates a file).

`create_feature_csv`

Creates a CSV file for feature importance results.

Parameters:

- `csv_filename` (str): Name of the CSV file to create.

Returns:

- None (creates a file).

`create_loss_csv`

Creates a CSV file to log model loss history during training.

Parameters:

- `csv_filename` (str): Name of the CSV file to create.

Returns:

- None (creates a file).

`write_model_results_to_csv`

Appends model evaluation results to a CSV file.

Parameters:

- `station` (str): Identifier for the station being evaluated.
- `model_name` (str): Name of the model.
- `config` (dict): Model configuration (features, input/output lengths).
- `metrics` (dict): Evaluation metrics (MAE, MSE, MAPE).
- `filename` (str): Name of the CSV file (default: 'fake_results.csv').

Returns:

- None (writes to the file).

`write_feature_results_to_csv`

Logs performance results for models after dropping specific features.

Parameters:

- `label_feature` (str): Label feature for the model.
- `dropped_feature` (str): Feature that was excluded.
- `model_name` (str): Name of the model.
- `metrics` (dict): Performance metrics after dropping the feature.
- `csv_filename` (str): Name of the CSV file to append results (default: 'feature_importance_results.csv').

Returns:

- None (writes to the file).

`calculate_original_performance`

Evaluates all models using the full feature set and logs their performance.

Parameters:

- `models` (dict): Dictionary of model names and objects.
- `config` (dict): Model configuration (features, input/output lengths).
- `train_df`, `val_df`, `test_df` (pd.DataFrame): Training, validation, and test datasets.

Returns:

- `original_performance` (dict): Baseline performance metrics for all models.

`drop_feature_and_evaluate`

Trains and evaluates models after dropping one feature at a time, measuring the impact.

Parameters:

- `config` (dict): Model configuration (features, input/output lengths).
- `original_performance` (dict): Baseline performance metrics for comparison.
- `train_df`, `val_df`, `test_df` (pd.DataFrame): DataFrames for training, validation, and testing.
- `features` (list): List of features to drop.

- `target` (str): Target variable.
- `CONV_WIDTH` (int): Convolution window size.
- `model_dir` (str): Directory to save models and results.

Returns:

- `feature_importance` (dict): Impact of each feature on model performance.

Precipitation Chance Forecasting

Introduction

This project aims to forecast precipitation chances at various hourly intervals (e.g., 1, 3, 6, 12, and 24 hours) using a combination of rule-based methods and machine-learning models. By first generating synthetic precipitation chance percentages through a rule-based system and then feeding this enriched data into linear and advanced machine learning models, the goal is to provide nuanced and accurate precipitation probability forecasts.

Rule-Based Forecasting Generation

A rule-based system was developed to generate initial precipitation chance percentages. This system relied on meteorological predictors such as precipitation levels, relative humidity, solar radiation, and temporal patterns to infer probabilities. The key steps included:

1. Feature Engineering: Relevant features capturing weather dynamics were derived, including indicators of cloud cover and trends in recent precipitation.
2. Threshold-Based Probability Assignment: Probabilities were computed using predefined thresholds to ensure variability, with a minimum baseline of 1–10% chance applied even in low-rain scenarios to reflect inherent uncertainties.
3. Interval Scaling: Probabilities were scaled according to the forecast interval, assigning higher chances for shorter intervals (e.g., 1-hour) and lower chances for longer intervals (e.g., 24-hour).

This rule-based approach was not intended to be predictive on its own but rather to generate synthetic probabilities that serve as input data for machine learning models.

Linear Model for Enhanced Forecast Generation

The synthetic data generated by the rule-based system was fed into a linear regression model. The purpose of this step was to introduce variation and context for the model to learn from, enabling it to produce more accurate and context-sensitive precipitation chance percentages. The steps included:

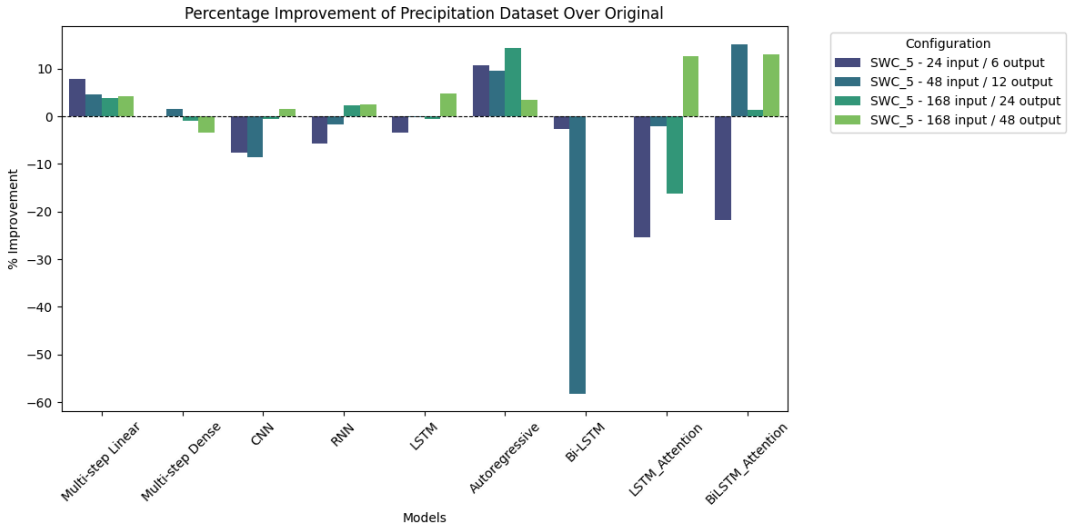
1. Integration of Rule-Based Data: The rule-based outputs were treated as features, alongside other meteorological predictors, to train the linear regression model.
2. Refinement of Probabilities: The linear model used the enriched dataset to refine precipitation forecasts, capturing relationships between features that the rule-based system could not model directly.

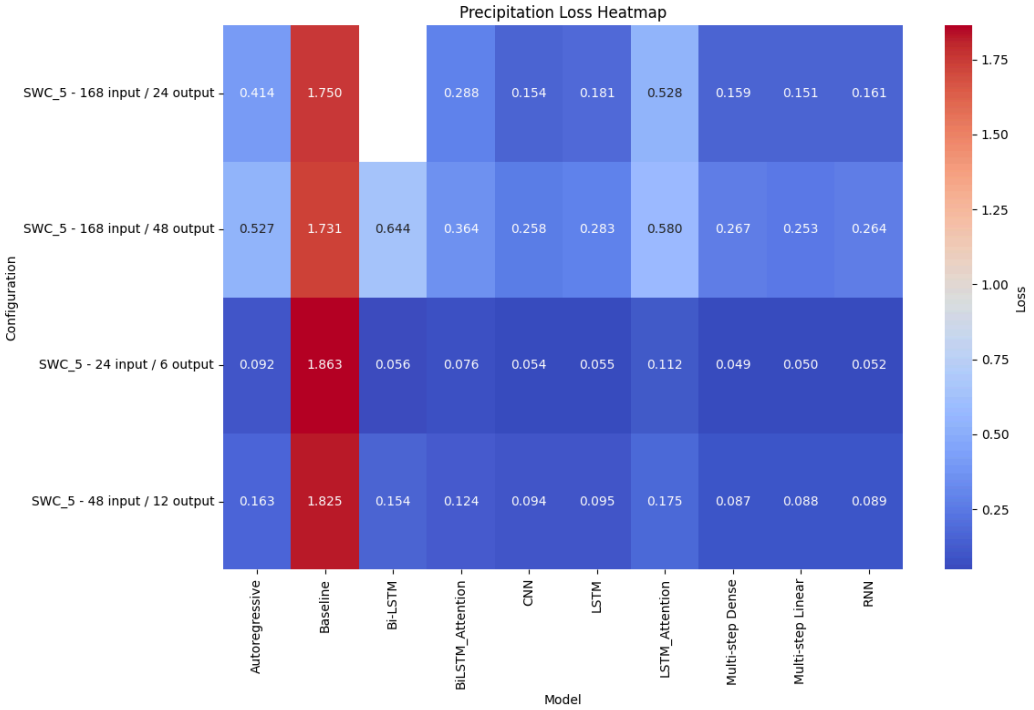
This combination provided the machine learning models with more robust inputs, resulting in forecasts with improved interpretability and variation.

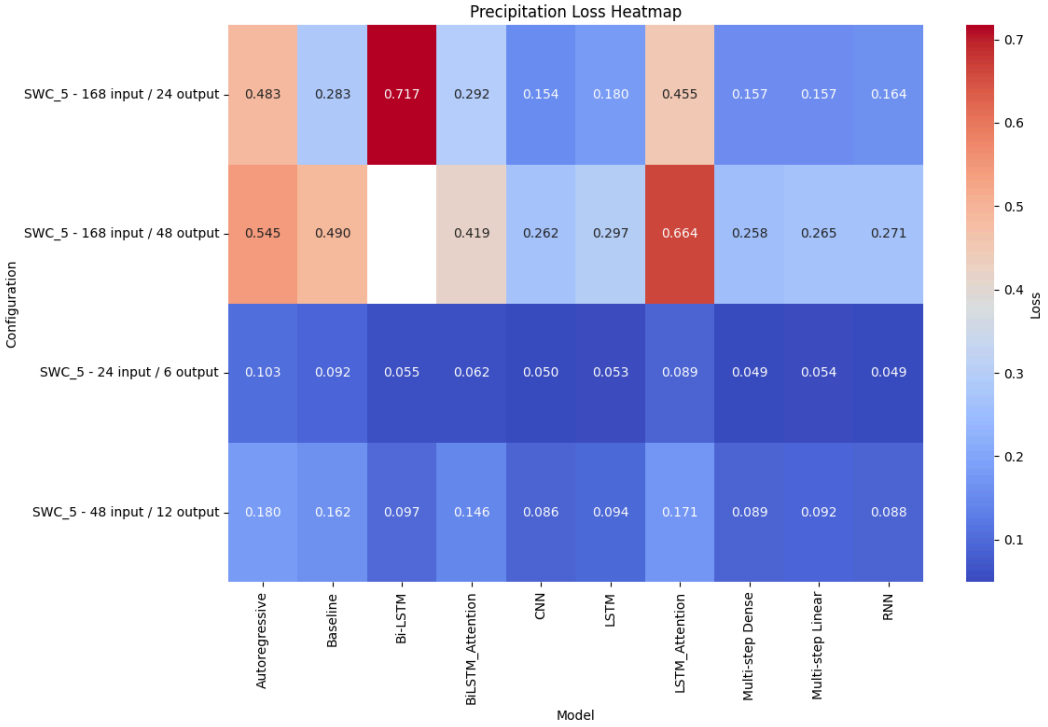
Conclusion/Results

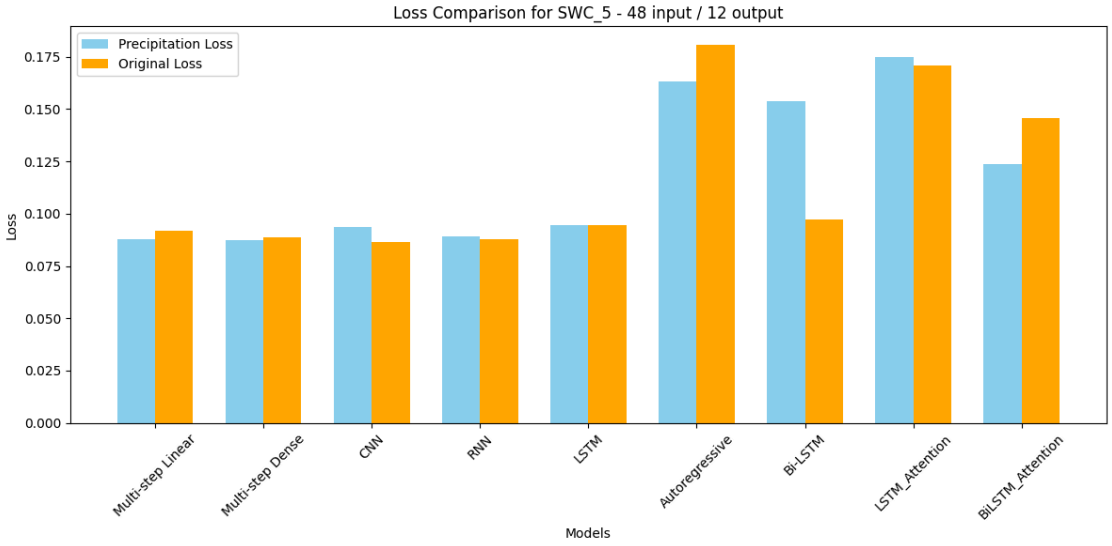
The use of a rule-based system as a preprocessing step for generating synthetic precipitation probabilities, followed by training a linear regression model on this enriched data, represents an ensemble method to generate more context and variation for the model to learn from. By combining deterministic and data-driven methodologies, this framework provides accurate, interpretable, and adaptable forecasts. Future work involves incorporating advanced machine learning models such as LSTMs and attention mechanisms to further enhance performance across varying forecast intervals.

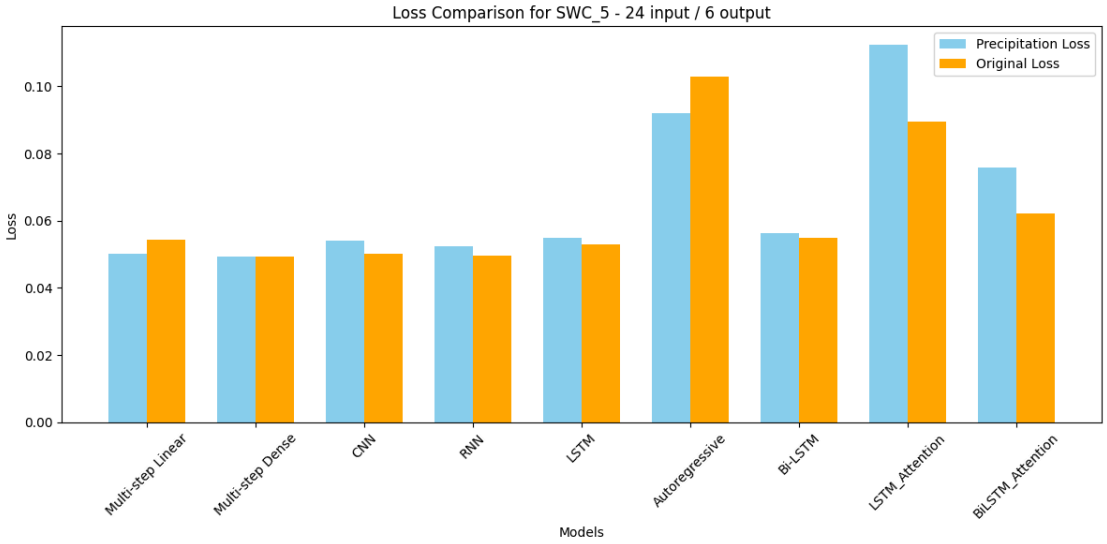
With the new forecasting features, the aim was to include these features in the original soil moisture dataset in order to improve accuracy and precision when predicting soil moisture content. The following charts show the results after comparing the original dataset with the new dataset including the precipitation forecasting. The comparison was based on the models from Alex and I's model comparison code and models. While there wasn't a clear improvement across all of the models, in some of the models, the new precipitation forecasting model performed better. The following charts show this improved performance. More detailed charts can also be found in the precipitation_comparison.ipynb.

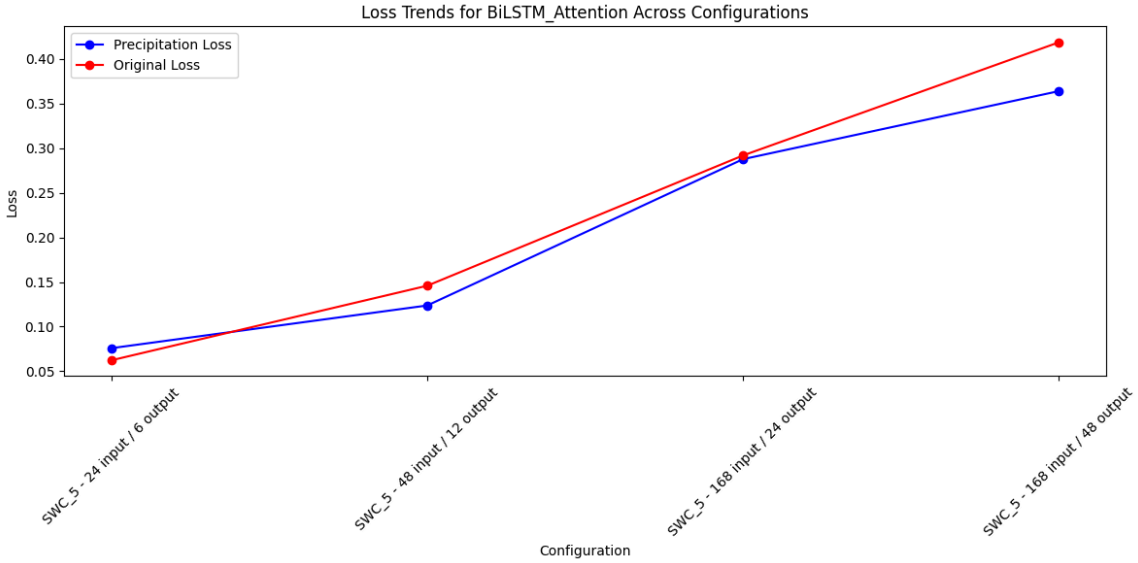



Precipitation Loss Heatmap
SWC_5 - 168 input / 24 output
SWC_5 - 168 input / 48 output
SWC_5 - 24 input / 6 output
SWC_5 - 48 input / 12 output
Configuration
0.414 1.750 0.288 0.154 0.181 0.528 0.159 0.151 0.161
0.527 1.731 0.644 0.364 0.258 0.283 0.580 0.267 0.253 0.264
0.092 1.863 0.056 0.076 0.054 0.055 0.112 0.049 0.050 0.052
0.163 1.825 0.154 0.124 0.094 0.095 0.175 0.087 0.088 0.089
Autoregressive
Baseline
Bi-LSTM
BiLSTM_Attention
CNN
LSTM
LSTM_Attention
Multi-step Dense
Multi-step Linear
RNN
Model
Loss
1.75
1.50
1.25
1.00
0.75
0.50
0.25


Precipitation Loss Heatmap
SWC_5 - 168 input / 24 output
SWC_5 - 168 input / 48 output
SWC_5 - 24 input / 6 output
SWC_5 - 48 input / 12 output
Configuration
0.483 0.283 0.717 0.292 0.154 0.180 0.455 0.157 0.157 0.164
0.545 0.490 0.419 0.262 0.297 0.664 0.258 0.265 0.271
0.103 0.092 0.055 0.062 0.050 0.053 0.089 0.049 0.054 0.049
0.180 0.162 0.097 0.146 0.086 0.094 0.171 0.089 0.092 0.088
Autoregressive
Baseline
Bi-LSTM
BiLSTM_Attention
CNN
LSTM
LSTM_Attention
Multi-step Dense
Multi-step Linear
RNN
Model
Loss
0.7
0.6
0.5
0.4
0.3
0.2
0.1


Loss Comparison for SWC_5 - 48 input / 12 output
Precipitation Loss
Original Loss
Loss
0.175
0.150
0.125
0.100
0.075
0.050
0.025
0.000
Multi-step Linear
Multi-step Dense
CNN
RNN
LSTM
Autoregressive
Bi-LSTM
LSTM_Attention
BiLSTM_Attention
Models


Loss Comparison for SWC_5 - 24 input / 6 output
Precipitation Loss
Original Loss
Loss
0.10
0.08
0.06
0.04
0.02
0.00
Multi-step Linear
Multi-step Dense
CNN
RNN
LSTM
Autoregressive
Bi-LSTM
LSTM_Attention
BiLSTM_Attention
Models


Loss Trends for BiLSTM_Attention Across Configurations
Precipitation Loss
Original Loss
Loss
0.40
0.35
0.30
0.25
0.20
0.15
0.10
0.05
SWC_5 - 24 input / 6 output
SWC_5 - 48 input / 12 output
SWC_5 - 168 input / 24 output
SWC_5 - 168 input / 48 output
Configuration